



EXPENSE TRACKER

SUBMITTED BY - SHANTANU YADAV
REGISTRATION NUMBER - 26BAI10689
ACADEMIC YEAR - 2026-27
SLOT - B21+B22+B23+E14+E21+E22

INTRODUCTION

An **Expense Tracker** is a simple Python project used to record and manage daily expenses. It helps users keep track of how much money they spend and where they spend it.

In this project, the user can **add an expense, view all expenses, and calculate the total amount spent**. Expenses can include things like food, travel, shopping, and other daily needs.

The main purpose of this project is to make expense management **simple, organized, and easy to understand** while practicing basic Python concepts such as **variables, lists, loops, conditions, functions, and user input**.

PROBLEM STATEMENT

Managing daily expenses manually can be difficult because people may forget where and how much money they have spent. Without proper tracking, it becomes hard to understand total spending and control unnecessary expenses.

The **Expense Tracker** project aims to provide a simple way to record daily expenses and calculate the total amount spent. The

system allows the user to enter details such as the expense name, category, and amount, and then view the recorded expenses.

The project helps users organize their spending, monitor expenses, and develop better money-management habits using a simple Python program.

OBJECTIVE

- To record daily expenses in a simple way.
- To keep track of the amount spent.
- To categorize expenses such as food, travel, shopping, etc.
- To calculate the total expenses.
- To display all recorded expenses clearly.
- To help users understand their spending habits.
- To practice basic Python concepts like variables, lists, loops, conditions, and functions.

FUNCTIONAL REQUIREMENT

The system should allow the user to add a new expense.

The system should allow the user to enter the expense name, category, and amount.

The system should store the entered expenses during the program.

The system should display all recorded expenses.

The system should calculate and display the total amount spent.

The system should allow the user to exit the program.

The system should show a simple menu of available options to the user.

The system should handle invalid inputs and ask the user to enter the correct information.

NON FUNCTIONAL REQUIREMENT:

The Expense Tracker should be simple, user-friendly, and easy to understand so that anyone can use it without needing advanced computer knowledge. The system should respond quickly when the user enters information or selects an option. It should provide

accurate calculations of the total expenses and display the information clearly.

The program should be **reliable and stable**, meaning it should continue working properly and handle incorrect inputs without suddenly stopping. The source code should be **easy to read, maintain, and modify**, allowing new features to be added later. The application should also require minimal system resources and run smoothly on a computer with Python installed. User-entered expense information should be handled **privately and securely** within the program.

SYSTEM ARCHITECTURE:

The Expense Tracker follows a simple architecture where the user interacts with the Python program through a menu-driven interface. The system takes expense details as input, processes them, stores them temporarily, and displays the required information to the user.

The architecture can be divided into three main parts:

1. User Interface:

The user interacts with the program through a simple menu. The

user can choose options such as adding an expense, viewing expenses, calculating the total amount, or exiting the program.

2. Processing:

The Python program receives the user's input and processes the expense information. It checks the entered values, stores the expense details, and calculates the total amount spent using basic Python operations.

3. Data Storage:

The entered expenses are stored in a Python list while the program is running. Each expense contains basic information such as the expense name, category, and amount.

Simple Architecture Flow

User → Menu → Input Expense → Process Data → Store Expense → Display Expenses / Calculate Total → User

DESIGN DECISION & RATIONALE

The Expense Tracker is designed as a simple menu-driven Python program so that it is easy to understand and use. The design focuses on basic Python concepts rather than advanced modules or complicated features.

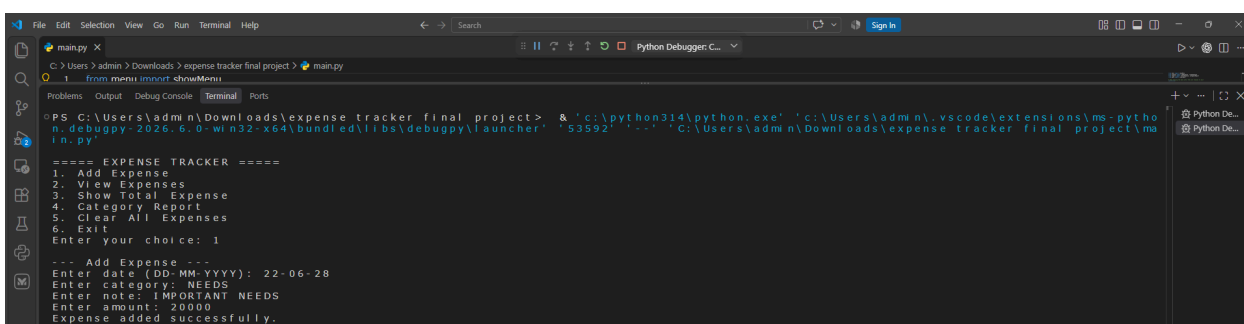
The program uses a list to store expenses temporarily because a list is simple and suitable for handling multiple expense records. Each expense can contain details such as the expense name, category, and amount.

A menu-based system is used so the user can easily select different operations, such as adding an expense, viewing expenses, calculating the total, or exiting the program. Basic functions are used to separate different tasks, which makes the code easier to read and modify.

The system uses simple input validation to reduce errors when the user enters incorrect information. The design avoids unnecessary advanced features and focuses on making the project simple, clear, reliable, and suitable for beginners learning Python.

SCREENSHOTS AND RESULTS

1. ADDING EXPENSE



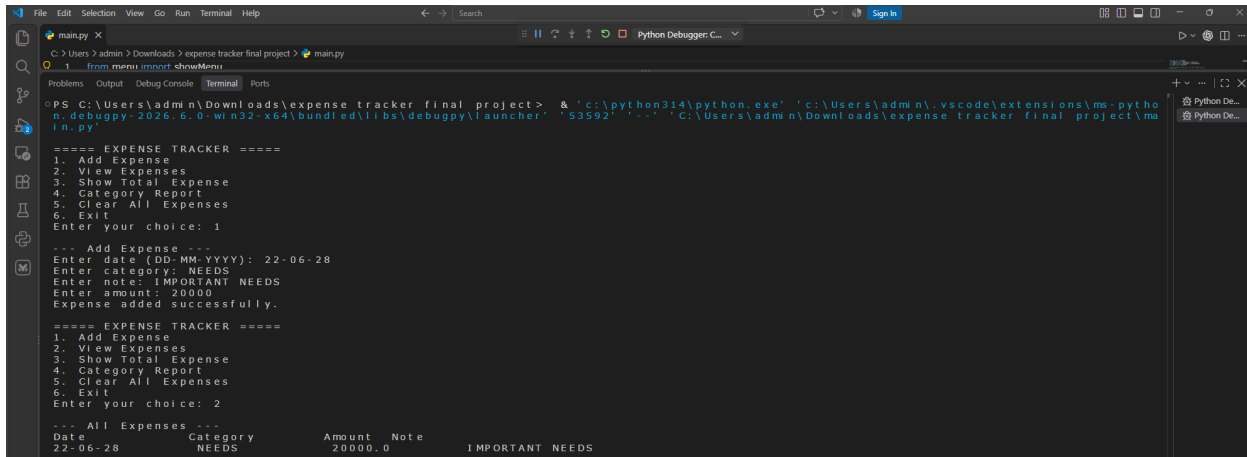
```
main.py X
C:\Users\admin> Downloads > expense tracker final project > main.py
from menu import showMenu

PS C:\Users\admin\Downloads\expense tracker final project> & 'c:\python314\python.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53592' '-' 'C:\Users\admin\Downloads\expense tracker final project\main.py'

==== EXPENSE TRACKER ====
1. Add Expense
2. View Expenses
3. Show Total Expense
4. Category Report
5. Clear All Expenses
6. Exit
Enter your choice: 1

--- Add Expense ---
Enter date (DD-MM-YYYY): 22-06-28
Enter category: NEEDS
Enter notes: IMPORTANT NEEDS
Enter amount: 20000
Expense added successfully.
```

2. VIEW EXPENSE



The screenshot shows a VS Code editor with a terminal window open. The terminal displays the output of a Python script. The user has selected option 2, 'View Expenses'. The program then prompts for a date, category, and amount. The user enters '22-06-28', 'NEEDS', and '20000'. The program then displays a table of expenses.

```
PS C:\Users\admin\Downloads\expense_tracker_final_project> & 'c:\python314\python.exe' 'c:\Users\admin\.vscode\extensions\ms-python-deb...
main.py
1 from menu import showMenu

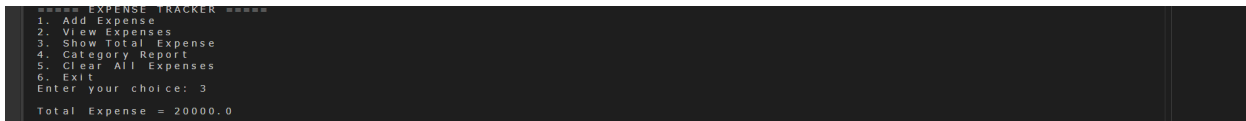
===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Show Total Expense
4. Category Report
5. Clear All Expenses
6. Exit
Enter your choice: 1

--- Add Expense ---
Enter date (DD-MM-YYYY): 22-06-28
Enter category: NEEDS
Enter notes: IMPORTANT NEEDS
Enter amount: 20000
Expense added successfully.

===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Show Total Expense
4. Category Report
5. Clear All Expenses
6. Exit
Enter your choice: 2

--- All Expenses ---
Date      Category      Amount  Note
22-06-28   NEEDS         20000.0  IMPORTANT NEEDS
```

3. SHOW TOTAL EXPENSE



The screenshot shows a terminal window with the output of the 'SHOW TOTAL EXPENSE' functionality. The user has selected option 3, 'Show Total Expense'. The program then displays the total expense.

```
===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Show Total Expense
4. Category Report
5. Clear All Expenses
6. Exit
Enter your choice: 3

Total Expense = 20000.0
```

4. CATEGORY REPORT



The screenshot shows a terminal window with the output of the 'CATEGORY REPORT' functionality. The user has selected option 4, 'Category Report'. The program then displays the category report.

```
===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Show Total Expense
4. Category Report
5. Clear All Expenses
6. Exit
Enter your choice: 4

--- Category Report ---
NEEDS = 20000.0
```

5. CLEAR ALL EXPENSE



The screenshot shows a terminal window with the output of the 'CLEAR ALL EXPENSE' functionality. The user has selected option 5, 'Clear All Expenses'. The program then prompts for confirmation to clear all expenses. The user enters 'NO', and the program displays a message indicating that the data was not deleted.

```
===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Show Total Expense
4. Category Report
5. Clear All Expenses
6. Exit
Enter your choice: 5
Are you sure you want to clear all expenses? (yes/no): NO
Data was not deleted.
```

6. EXIT



The screenshot shows a terminal window with the output of the 'EXIT' functionality. The user has selected option 6, 'Exit'. The program then displays a message indicating that the user has exited the application.

```
===== EXPENSE TRACKER =====
1. Add Expense
2. View Expenses
3. Show Total Expense
4. Category Report
5. Clear All Expenses
6. Exit
Enter your choice: 6
Thank you for using Expense Tracker.
PS C:\Users\admin\Downloads\expense_tracker_final_project>
```

CHALLENGES FACED

- Understanding how to take user input correctly.
- Storing multiple expenses using a Python list.
- Calculating the total amount of expenses accurately.
- Designing a simple and easy-to-use menu.
- Handling invalid or incorrect user inputs.
- Displaying all recorded expenses clearly.
- Using loops and conditions correctly.
- Keeping the program simple and beginner-friendly.
- Organizing the code using basic functions.
- Testing the program and fixing small errors.

LEARNING & KEY TAKEAWAYS

- 1.Learned the basics of Python programming.
- 2.Understood the use of variables, lists, loops, and conditions.
- 3.Learned how to take and process user input.
- 4.Understood how to create and use functions.
- 5.Learned how to store and manage expense details.
- 6.Improved logical thinking and problem-solving skills.
- 7.Learned how to perform calculations using Python.

8. Understood the importance of input validation.
9. Gained experience in testing and debugging a program.
10. Learned how to create a simple menu-driven application.

FUTURE ENHANCEMENT

Future Enhancements

- Add a feature to edit or delete expenses.
- Add date and time for each expense.
- Add different expense categories such as food, travel, shopping, and education.
- Add a feature to set a monthly budget.
- Show the remaining budget after expenses.
- Add monthly and yearly expense summaries.
- Add simple charts or graphs to understand spending habits.
- Save expenses permanently using a file or database.
- Add a search and filter option for expenses.
- Develop a GRAPHIC USER INTERFAC or web version for easier use.
- Add a feature to generate expense reports.

REFERENCE

1.VIDYARTHI- BUILD YOUR OWN PROJECT, GENERAL PROJECT INSTRUCTIONS & SUBMISSION GUIDELINES

2. PYTHON DOCUMENTATION, PYTHON PROGRAMMING LANGUAGE DOCUMENTATION

3. Python by - NICHOLA LACEY